

CSE 4345: Big Data Analytics

Week 4, Part 2 — Research Article & Case Study

Rokan Uddin Faruqui

Professor

Department of Computer Science & Engineering

University of Chittagong, Chittagong

CSE, PUC, Spring 2026

Today's Agenda

- 1 Paper 1 — Dean & Ghemawat (2004): MapReduce
- 2 Paper 2 — Chang et al. (2006): Bigtable
- 3 Case Study — Facebook Data Infrastructure (2008–2012)
- 4 Live Exercise

Case Study

Facebook Data Infrastructure (2008–2012): Hive on MapReduce for 30 PB analytics + HBase for real-time Messenger at 1 billion messages/day.

Research Articles Covered

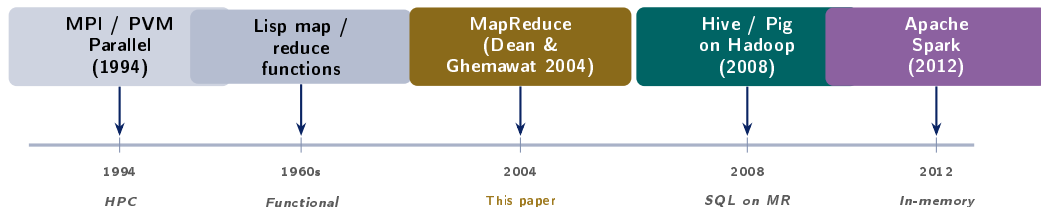
- 1 Dean, J., & Ghemawat, S. (2004). *MapReduce: Simplified Data Processing on Large Clusters*. OSDI 2004.
- 2 Chang, F., et al. (2006). *Bigtable: A Distributed Storage System for Structured Data*. OSDI 2006.

CLO Addressed

CLO1 — Understand distributed processing & storage
PLO(a,b)

Paper 1 — Dean & Ghemawat (2004): MapReduce

Research Landscape: Parallel Processing Timeline



Why MapReduce Was Disruptive

MPI required programmers to handle **process spawning**, **message passing**, and **failure recovery** manually. MapReduce hid all of this behind two functions — **map** and **reduce** — and handled fault tolerance, data locality, and load balancing *automatically*.

Dean & Ghemawat (2004): Publication Context

Full Citation

Dean, J., & Ghemawat, S. (2004). **MapReduce: Simplified Data Processing on Large Clusters**. In *Proceedings of the 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI '04)*, pp. 137–150. San Francisco, CA, USA.

Venue: USENIX OSDI — Rank A* (CORE)

Cited by: >21,000 (Google Scholar, 2024)

Authors: Jeffrey Dean & Sanjay Ghemawat (Google Systems Research Group)

The Motivation (from the paper)

Google engineers were writing hundreds of special-purpose programs to process large datasets. After noticing that *most programs followed the same pattern*, Dean and Ghemawat abstracted it into a single framework. The first internal production use: **building the web search inverted index**.

The Programming Model (1 sentence)

`map(k1,v1) → list(k2,v2)`

`reduce(k2, list(v2)) → list(v3)`

Classic Example: Word Count

Map: for each word w in document, emit $(w,1)$

Shuffle: group all $(w,1)$ pairs by key w

Reduce: sum the list of 1s for each w

⇒ Output: (w, count) for all w

Technical Contribution 1 — Execution Model & Data Locality

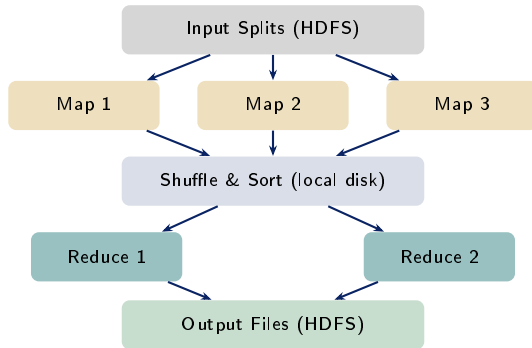
MapReduce Execution Flow

- ➊ **Split**: input file divided into M splits (default 64–128 MB, one split per mapper)
- ➋ **Map phase**: M map tasks run in parallel on nodes that *hold the input splits* (**data locality** — compute goes to data)
- ➌ **Shuffle & Sort**: map output key-value pairs **partitioned** by key, written to local disk, then **sorted** and transferred to reducers
- ➍ **Reduce phase**: R reduce tasks each receive all values for a key range; write output to GFS/HDFS

Data Locality Optimisation

The master scheduler **prefers to assign map tasks to nodes that already hold the corresponding HDFS block**. This avoids network transfer of the (large) input data. Only the (smaller) map output crosses the network.

Technical Contribution 1 — Execution Model & Data Locality



Technical Contribution 2 — Fault Tolerance & Stragglers

Task-Level Fault Tolerance

- Master **pings** each worker periodically; worker deemed dead after no response for 5 minutes
- All **completed map tasks** on a dead worker are **re-executed** (output was on local disk, now inaccessible)
- Completed **reduce tasks** are *not* re-executed (output is in GFS/HDFS, fault-tolerant already)
- A task is **in-progress** on a dead worker is automatically rescheduled on another worker
- **Determinism requirement:** map/reduce functions must produce the same output for the same input — this makes re-execution semantically correct

Master Fault Tolerance

Master writes **periodic checkpoints** of its data structures. On failure, a new master restores from the last checkpoint. In practice, Google reported that master failure was rare and required manual restart.

Technical Contribution 2 — Fault Tolerance & Stragglers

Straggler Mitigation: Backup Tasks

A **straggler** is a map or reduce task that takes far longer than its peers — caused by a slow disk, bad network, or noisy CPU.

Solution: when a MapReduce job is near completion, the master launches *backup copies* of all in-progress tasks. Whichever copy finishes first is accepted; the other is killed.

Result (from the paper): backup tasks reduce job completion time by up to **44%** on a 200-task job with one slow machine.

Combiner Function

Optional **combiner** runs on the mapper node *before* the shuffle — performs partial aggregation. In word count: sum local counts before sending to reducers. Reduces network transfer by up to **10×**.

Direct Descendants & Legacy

- **Hadoop MapReduce** (2006) — open-source Java reimplementation; shipped by Yahoo!, Cloudera, Hortonworks
- **Apache Hive** (2009) — SQL-to-MapReduce compiler; Facebook's solution for analysts who can't write Java MR
- **Apache Pig** (2008) — dataflow language compiling to MapReduce; Yahoo!'s ETL automation tool
- **Apache Spark** (2010) — replaced MapReduce's disk-based shuffle with in-memory RDDs; Spark's DAG model is a direct response to MR's limitations
- **Google's Dataflow / Flume** (2010+) — Google's own successor, which led to Apache Beam (Week 11)
- **AWS EMR, Azure HDInsight** — cloud-managed Hadoop MapReduce as a service; still widely used in 2024

Paper 2 — Chang et al. (2006): Bigtable

Chang et al. (2006): Publication Context & Data Model

Full Citation

Chang, F., Dean, J., Ghemawat, S., Hsieh, W. C., Wallach, D. A., Burrows, M., Chandra, T., Fikes, A., & Gruber, R. E. (2006). **Bigtable: A Distributed Storage System for Structured Data**. In *Proceedings of the 7th USENIX OSDI*, pp. 205–218. Seattle, WA, USA.

Venue: USENIX OSDI — Rank A*

Cited by: >13,000 (Google Scholar, 2024)

Products using Bigtable: Google Search, Google Maps, Gmail, Google Analytics, YouTube

The Data Model (2 lines)

A Bigtable is a sparse, distributed, persistent multi-dimensional sorted map.

Key: (row key, column key, timestamp) → byte array value

Webtable: Storing the Web

Row key	Column	Value
com.cnn.www	content:	<html>...
com.cnn.www	anchor:cnnsi	"CNN"
com.cnn.www	anchor:my.look	"CNN.com"
com.bbc.www	content:	<html>...

- Row keys in **reverse-domain order**: groups pages from same domain together
- Each row access is **atomic**
- Column families (content, anchor) must be declared; columns within a family are dynamic
- Multiple **timestamped versions** per cell

Bigtable Architecture — Tablets, SSTable, Compaction

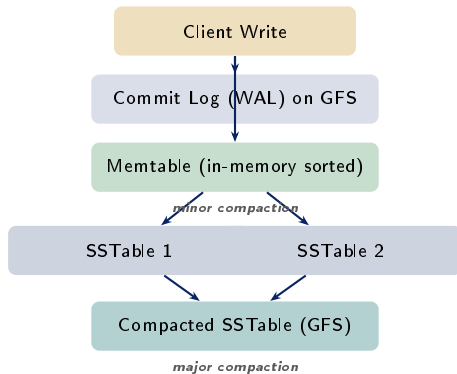
Three-Layer Architecture

- **Master**: assigns tablets to tablet servers, detects failures, load-balances, manages schema changes. Never on the read/write data path.
- **Tablet servers** (100s–1000s): serve reads and writes for their assigned tablets (each tablet is a contiguous range of rows, 100 MB–200 MB).
- **Chubby** (distributed lock service): Bigtable uses Chubby for master election, tablet server registration, and schema storage. Chubby is built on Paxos consensus.

Write Path: Memtable + WAL + SSTable

- 1 Write appended to **commit log** (WAL)
- 2 Write inserted into in-memory **memtable** (sorted by key)
- 3 When memtable exceeds threshold: frozen, converted to an immutable **SSTable** on GFS
- 4 **Compaction**: periodic merge of SSTables to reduce read amplification and reclaim deleted cells

Bigtable Architecture — Tablets, SSTable, Compaction



Bigtable vs. HBase — Design Correspondence

Concept	Bigtable (Google, 2006)	HBase (Apache, 2008)
Storage layer	GFS	HDFS
Coordination	Chubby (Paxos)	ZooKeeper (ZAB protocol)
Tablet unit	Tablet (100–200 MB)	Region (256 MB default)
Tablet server	Tablet server	RegionServer
Master	Bigtable master	HBase Master
In-memory store	Memtable	MemStore
Sorted file format	SSTable	HFile (SSTable-compatible)
Compaction	Minor + Major	Minor + Major
Bloom filters	Per SSTable	Per HFile
Language	C++	Java (JVM)

Key Lesson

Case Study — Facebook Data Infrastructure (2008–2012)

Facebook's Data Scale in 2008

The Analytics Challenge

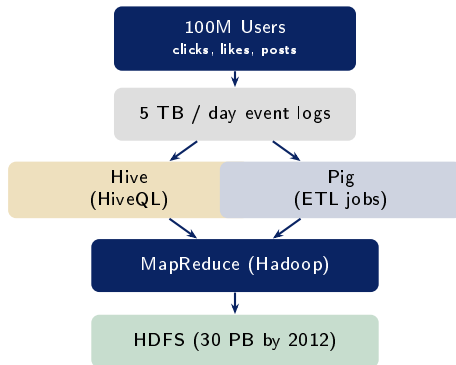
By 2008 Facebook had:

- **100 million active users** producing **15 billion page-views per day**
- **5 TB/day** of event logs (clicks, impressions, social graph updates) that needed daily aggregation for ad billing and product analytics
- A team of analysts and product managers who needed **SQL-like queries** over those logs — but could not write Java MapReduce
- Oracle data warehouses that could not handle **week-over-week log accumulation** (reaching petabyte scale by 2010)

The Decision

Facebook deployed Hadoop in 2007, initially for log processing. When analysts demanded SQL access, engineers **Joydeep Sen Sarma** and **Ashish Thusoo** built **Hive** — a SQL-to-MapReduce compiler. Hive was open-sourced at VLDB 2009.

Facebook's Data Scale in 2008



HBase at Facebook — Real-Time Messenger (2010)

The Real-Time Problem

While MapReduce served batch analytics, Facebook Messenger (launched 2008) required a **real-time** message store:

- **1 billion messages/day** at peak
- Messages must be **written and read in <100 ms** (MapReduce batch latency: hours)
- **High write throughput** (users send bursts); **random point reads** (fetch inbox for a user)
- Message threads stored as rows: row key = user_id + thread_id; each message as a column with timestamp version

HBase: The Right Tool

Facebook chose HBase (Bigtable clone) over MySQL sharding because: **(1)** automatic tablet splitting handles hotspots; **(2)** memtable absorbs write bursts at microsecond latency; **(3)** column families allow efficient thread-level scans.

HBase at Facebook — Real-Time Messenger (2010)

Scale Numbers (Facebook 2010 HBase deployment)

Metric	Value
HBase cluster nodes	100
Messages stored	>135 billion
Peak write rate	1M rows / sec
Median read latency	<10 ms
Row key space	user $\times$ thread
Replication factor	3 (via HDFS)
Daily compaction volume	$\sim$ 1 TB

Open-Source Contribution

Facebook's Hadoop + Hive + HBase deployment was documented in Thusoo et al. (VLDB 2009 & 2010) and drove >500 patches to Apache Hadoop / HBase between 2008–2012.

Key Lessons from Facebook's Case

- 1 **Right tool for the workload:** MapReduce for batch analytics, HBase for random-access real-time reads — a single database cannot serve both optimally
- 2 **SQL lowers the barrier:** Hive's HiveQL let hundreds of non-Java analysts query petabyte-scale data; tool ergonomics matter as much as raw performance
- 3 **Schema flexibility has a cost:** HBase's schemaless column families allowed rapid iteration, but required careful key design to avoid hotspotting
- 4 **Compaction is a hidden cost:** HBase compaction consumed significant I/O; Facebook had to schedule major compactions during off-peak hours
- 5 **Open source as a talent strategy:** open-sourcing Hive attracted 100+ external contributors and made it an industry standard within 3 years

Discussion Questions (CLO1, CLO4)

- 1 MapReduce's shuffle phase requires **all map tasks to complete** before reducers start. Explain why this is a *blocking barrier* and describe how Apache Spark's **narrow vs. wide transformations** eliminate some of these barriers. [CLO1 — Understand]
- 2 Facebook's HBase row key is `user_id + thread_id`. If all active users are in the range 1000-1999, identify the hotspot problem and propose a **row key salting strategy** to distribute load evenly across RegionServers. [CLO4 — Apply]
- 3 Bigtable uses **Chubby** (Paxos) while HBase uses **ZooKeeper** (ZAB). Compare the two coordination protocols and explain which provides stronger consistency guarantees. [CLO1 — Analyse]

Live Exercise

Live Exercise — MapReduce Word Count in Python (mrjob)

Task (20 minutes, pairs)

Run a MapReduce word count using **mrjob** (Python MapReduce framework) locally — no Hadoop cluster required.

Goal: count word frequencies in any text file and find the top-10 most frequent words.

Extension 1: add a **combiner** method and measure whether it reduces intermediate output size.

Extension 2: modify to compute **average word length** per first letter (group-by = first_letter, aggregate = mean(len(word))).

Install

```
pip install mrjob
```

Live Exercise — MapReduce Word Count in Python (mrjob)

mrjob Word Count Skeleton

```
from mrjob.job import MRJob
from mrjob.step import MRStep

class WordCount(MRJob):

    def steps(self):
        return [MRStep(
            mapper=self.mapper,
            combiner=self.combiner,
            reducer=self.reducer)]

    def mapper(self, _, line):
        for word in line.split():
            yield word.lower(), 1

    def combiner(self, word, counts):
        yield word, sum(counts)

    def reducer(self, word, counts):
        yield word, sum(counts)

if __name__ == '__main__':
    WordCount.run()
```

Research Articles

- Dean, J., & Ghemawat, S. (2004). MapReduce: Simplified data processing on large clusters. *Proceedings of the 6th USENIX OSDI*, 137–150.
- Chang, F., Dean, J., Ghemawat, S., et al. (2006). Bigtable: A distributed storage system for structured data. *Proceedings of the 7th USENIX OSDI*, 205–218.

Textbooks [TW], [SA], [TE]

- **[TW]** White, T. (2015). *Hadoop: The Definitive Guide*, 4th ed. (Ch. 2 — MapReduce; Ch. 5 — HBase). O'Reilly.
- **[SA]** Acharya, S., & Chellappan, S. (2015). *Big Data and Analytics* (Ch. 4 — MapReduce programming model). Wiley.
- **[TE]** Erl, T., Khattak, W., & Buhler, P. (2016). *Big Data Fundamentals* (Ch. 6 — NoSQL and column-family stores). Prentice Hall.

Key Takeaways

- MapReduce (2004) proved that **functional decomposition into map + reduce** enables automatic fault tolerance, data locality, and parallelism — turning distributed programming into a matter of writing two functions
- Bigtable (2006) showed that a **sparse, sorted multi-dimensional map** is the right primitive for structured data at web scale — HBase is its open-source twin
- Facebook's deployment demonstrated that **MapReduce (batch) and Bigtable/HBase (real-time)** are complementary, not competing, and most production architectures need both
- The **SSTable + memtable + compaction** pattern from Bigtable is now the foundation of RocksDB, Cassandra, and every modern LSM-tree storage engine

Quote of the Week

“MapReduce is a programming model for processing large data sets with a parallel, distributed algorithm on a cluster.”

— Dean & Ghemawat (2004)

Part 1 Preview — Week 5

YARN, File Formats (Parquet / ORC / Avro)

- YARN resource management model
- Columnar formats: Parquet vs. ORC
- Avro for schema evolution
- Format benchmarks: query speedup